# PL/pgSQL Functions & Exception Handling

Module 6 • Put trusted business logic close to the data.

MODULE

06

**90 minutes**

20 min Explain
15 min Demonstrate
40 min Hands-on
15 min Review

**developer_lab continues**

# Module 6 turns business rules into callable database logic

1

## ENCAPSULATE

Give repeated calculations and validations one tested implementation.

2

## CONTROL

Combine SQL with variables, decisions and precise return values.

3

## FAIL SAFELY

Reject invalid work with actionable messages and SQLSTATE codes.

**A function is an interface: its inputs, output and side effects must be deliberate.**

# By the end, learners can complete six tasks

01 Choose SQL or PL/pgSQL for a function

02 Define parameters and return types

03 Use variables, SELECT INTO and FOUND

04 Apply IF, CASE and RETURN QUERY

05 Raise and handle expected database errors

06 **Test a stock-reservation function safely**

SUCCESS = ATOMIC STOCK RESERVATION

# Use the simplest language that expresses the rule clearly

### LANGUAGE SQL

One query or expression
Planner-friendly inlining opportunities
Best for direct calculations and lookups
Little procedural ceremony

### LANGUAGE plpgsql

Multiple statements
Variables and branching
Loops and diagnostics
Structured error handling

**Do not add procedural code when one clear SQL statement is enough.**

# A function contract has five visible parts

```
CREATE OR REPLACE FUNCTION app.order_total(p_order_id bigint)
RETURNS numeric
LANGUAGE sql
STABLE
AS $function$
  SELECT COALESCE(SUM(quantity * unit_price), 0)
  FROM app.order_items
  WHERE order_id = p_order_id;
$function$;
```

## CONTRACT

Name + parameter types
Return type
Language

## BEHAVIOR

Volatility promise
Dollar-quoted body
Executable statements

# Call the function and test its boundary cases

```
SELECT app.order_total(1);

SELECT o.order_id, app.order_total(o.order_id)
FROM app.orders AS o
ORDER BY o.order_id;

SELECT app.order_total(999999); -- returns 0
```

### SCHEMA-QUALIFY

Use app.order_total so resolution is explicit.

### TEST THE CONTRACT

Existing order, empty order, unknown order and NULL input.

**Decide whether “not found” should return 0, NULL, no row or an exception.**

# Parameters make the call readable and predictable

```
CREATE FUNCTION app.discounted_price(
  p_price numeric,
  p_rate numeric DEFAULT 0.10
) RETURNS numeric
LANGUAGE sql IMMUTABLE
RETURN round(p_price * (1 - p_rate), 2);

SELECT app.discounted_price(100);
SELECT app.discounted_price(p_rate => 0.15, p_price => 100);
```

### DEFAULTS

Optional parameters must follow required input parameters.

### NAMED NOTATION

Names document the call and reduce ordering mistakes.

# Return shape determines how callers consume the result

| DECLARATION | SHAPE | CALL PATTERN |
| --- | --- | --- |
| **RETURNS numeric** | One scalar value | SELECT app.order_total(1) |
| **RETURNS app.products** | One composite row | SELECT (app.find_product(...)).* |
| **RETURNS TABLE (...)** | Named result columns | SELECT * FROM app.product_search(...) |
| **RETURNS SETOF type** | Zero or more rows | Use in the FROM clause |

**Prefer RETURNS TABLE when named columns make the caller clearer.**

# A PL/pgSQL block separates declarations from actions

```
CREATE OR REPLACE FUNCTION app.example(p_id bigint)
RETURNS text LANGUAGE plpgsql AS $function$
DECLARE
  v_status text;
BEGIN
  -- executable statements
  RETURN v_status;
EXCEPTION
  WHEN others THEN
    -- optional handler
    RAISE;
END;
$function$;
```

### DECLARE

Local variables and constants.

### BEGIN … END

Statements executed in order.

### EXCEPTION

Optional handlers for error conditions.

# SELECT INTO captures one query result

```
DECLARE
  v_stock integer;
BEGIN
  SELECT stock_qty INTO v_stock
  FROM app.products
  WHERE sku = p_sku;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'Unknown SKU: %', p_sku
      USING ERRCODE = 'P0002';
  END IF;
END;
```

### FOUND

True when the preceding relevant SQL statement affected or returned a row.

### STRICT OPTION

SELECT INTO STRICT requires exactly one row; zero or many rows raise an error.

# IF and CASE make business decisions explicit

```
IF p_qty IS NULL OR p_qty <= 0 THEN
  RAISE EXCEPTION 'Quantity must be positive';
ELSIF v_stock < p_qty THEN
  RAISE EXCEPTION 'Only % units remain', v_stock;
END IF;

v_label := CASE
  WHEN v_stock = 0 THEN 'OUT'
  WHEN v_stock < 5 THEN 'LOW'
  ELSE 'AVAILABLE'
END;
```

### IF / ELSIF

Use for actions that depend on conditions.

### CASE EXPRESSION

Use when one value is selected from several conditions.

**Validate inputs before changing data.**

# RETURN QUERY streams a query into a table result

```
CREATE FUNCTION app.low_stock(p_limit integer DEFAULT 5)
RETURNS TABLE(sku text, product_name text, stock_qty integer)
LANGUAGE plpgsql STABLE AS $function$
BEGIN
  RETURN QUERY
  SELECT p.sku, p.name, p.stock_qty
  FROM app.products AS p
  WHERE p.stock_qty <= p_limit
  ORDER BY p.stock_qty, p.sku;
END;
$function$;
```

### CALL

```
SELECT *
FROM app.low_stock(10);
```

### WATCH NAMES

Qualify table columns when output parameter names could be ambiguous.

# Prefer set-based SQL before writing a loop

### SET-BASED FIRST

UPDATE many matching rows once
Let the planner choose access paths
Fewer SQL/PL transitions
Usually shorter and faster

### LOOP WHEN NEEDED

Per-row procedural behavior
Dynamic statement sequence
Controlled retry or batching
Clear termination condition

**A loop is a control-flow tool—not a substitute for a relational query.**

# Volatility is a promise to PostgreSQL’s planner

### IMMUTABLE

Same arguments always produce the same result. No database lookups that can change.

### STABLE

May read database state but does not modify it; stable within one statement.

### VOLATILE

May change data or return different results; this is the default.

**Mark volatility honestly—an incorrect promise can produce incorrect plans or results.**

# Function security requires an explicit trust boundary

```
CREATE FUNCTION app.secure_report()
RETURNS TABLE (...)
LANGUAGE sql
SECURITY DEFINER
SET search_path = pg_catalog, app
AS $function$
  SELECT ...;
$function$;

REVOKE ALL ON FUNCTION app.secure_report() FROM PUBLIC;
```

### SECURITY INVOKER

Default: execute with the caller's privileges.

### SECURITY DEFINER

Execute as owner; lock search_path and grant narrowly.

**Use SECURITY DEFINER only when the privilege elevation is necessary and reviewed.**

# RAISE communicates severity, message and SQLSTATE

```
RAISE NOTICE 'Checking SKU %', p_sku;

RAISE EXCEPTION 'Insufficient stock for %', p_sku
  USING ERRCODE = 'P0001',
        DETAIL = format('requested=%s available=%s', p_qty, v_stock),
        HINT = 'Reduce the quantity or replenish stock';
```

### NOTICE / WARNING

Informational messages; execution can continue.

### EXCEPTION

Abort the current operation unless a matching handler catches it.

**Messages should help the caller act; SQLSTATE should help the caller classify.**

# Catch only errors you can handle meaningfully

```
BEGIN
  INSERT INTO app.customers(email, full_name)
  VALUES (p_email, p_name)
  RETURNING customer_id INTO v_customer_id;
EXCEPTION
  WHEN unique_violation THEN
    SELECT customer_id INTO v_customer_id
    FROM app.customers WHERE email = p_email;
END;
```

**HANDLED**

The block’s data changes roll back, then the handler runs.

**RE-RAISE**

Use bare RAISE; when the function cannot recover safely.

**Avoid WHEN others unless you re-raise or have a precise recovery policy.**

# Demo: reserve stock with one atomic UPDATE

```
CREATE OR REPLACE FUNCTION app.reserve_stock(
  p_sku text, p_qty integer
) RETURNS integer LANGUAGE plpgsql VOLATILE AS $function$
DECLARE v_left integer;
BEGIN
  IF p_qty IS NULL OR p_qty <= 0 THEN
    RAISE EXCEPTION 'Quantity must be positive' USING ERRCODE='22023';
  END IF;
  UPDATE app.products SET stock_qty = stock_qty - p_qty
  WHERE sku = p_sku AND stock_qty >= p_qty
  RETURNING stock_qty INTO v_left;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Unknown SKU or insufficient stock: %', p_sku
      USING ERRCODE='P0001';
  END IF;
  RETURN v_left;
END; $function$;
```

## WHY ATOMIC?

The predicate and decrement execute as one row-locking statement.

## OUTCOME

Success returns remaining stock. Failure changes nothing.

# Demo: prove success and failure behavior

```
BEGIN;
SELECT sku, stock_qty FROM app.products WHERE sku='KB-100';
SELECT app.reserve_stock('KB-100', 2) AS remaining;
SELECT sku, stock_qty FROM app.products WHERE sku='KB-100';
ROLLBACK;

SELECT app.reserve_stock('KB-100', 0);      -- 22023
SELECT app.reserve_stock('NO-SUCH-SKU', 1); -- P0001
SELECT app.reserve_stock('KB-100', 999999); -- P0001
```

**OBSERVE**

Valid call decrements once and returns the new balance.

**VERIFY**

ROLLBACK restores stock. Invalid calls expose distinct errors.

**Run destructive tests inside BEGIN … ROLLBACK.**

# Hands-on: build the function layer

01 Create app.order_total(bigint) as a SQL function

02 Create app.low_stock(integer) returning a table

03 Create app.reserve_stock(text, integer)

04 Raise 22023 for an invalid quantity

05 Raise P0001 when no stock row is updated

06 **Test all functions inside a rollback-safe transaction**

PAIR CHECK AFTER TASK 3

# Lab challenge: separate two failure causes

```
-- Improve reserve_stock so callers can distinguish:
-- 1) SKU does not exist
-- 2) SKU exists but stock is insufficient

-- Suggested design:
-- validate p_qty
-- attempt the atomic UPDATE
-- if NOT FOUND, test existence and raise a specific message
-- keep both failures free from partial changes
```

**REQUIREMENT**

Preserve the atomic stock decrement.

**EVIDENCE**

Show SQLSTATE, message and unchanged stock for each failure.

**Correctness first; avoid a read-then-write race.**

# Validate the lab with six checks

```
\df+ app.order_total
\df+ app.low_stock
\df+ app.reserve_stock

SELECT app.order_total(1);
SELECT * FROM app.low_stock(10);

BEGIN;
SELECT app.reserve_stock('KB-100', 1);
ROLLBACK;
```

## EXPECTED EVIDENCE

Three functions are listed
Return types are correct
Order total is numeric
Low-stock rows have named columns
Valid reservation returns an integer
Rollback restores the original stock

# Troubleshoot by matching symptom to cause

| SYMPTOM | LIKELY CAUSE | ACTION |
| --- | --- | --- |
| **function does not exist** | Argument types do not match | Cast input or inspect \df app.* |
| **column reference is ambiguous** | Output/variable name collides | Qualify table columns and aliases |
| **query returned more than one row** | SELECT INTO STRICT matched many | Fix predicate or return a set |
| **current transaction is aborted** | Earlier error was not rolled back | ROLLBACK, then fix the first error |

**Fix the first reported error; later messages are often consequences.**

# Knowledge check: explain the implementation choice

### SCENARIO A

One SELECT calculates an order total.

SQL or PL/pgSQL—and why?

### SCENARIO B

A stock rule validates, updates and returns a balance.

What must remain atomic?

### SCENARIO C

A handler catches every error and returns NULL.

What risk does this create?

**Suggested: SQL • conditional UPDATE + decrement • hidden failures and partial assumptions.**

# Module 6 takeaway: design the contract, then the control flow

1

## CONTRACT

Choose clear parameters, return shape, language and volatility.

2

## CORRECTNESS

Prefer set-based, atomic statements and validate before mutation.

3

## FAILURE

Raise actionable errors and catch only conditions you can recover from.

**Next: Module 7 — triggers, partitioning and full-text search.**